UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

TRABAJO DE INVESTIGACIÓN

Desarrollo y verificación de PANVIZ IRAS para la calidad de datos y la trazabilidad del software

Sistema de vigilancia de infecciones respiratorias agudas

AUTOR(ES):

- AJRA HUACSO, JEANS ANTHONY
- HANCCO MULLISACA, SERGIO DANILO
- HUACANI JARA, DENISE ANDREA
- VALDIVIA SEGOVIA RYAN FABIAN

DOCENTE:
Ing. Jesús Martín Silva Fernández

CURSO:
Calidad de Software

LUGAR:
AREQUIPA, PERÚ

FECHA:
10 de noviembre de 2026

RESUMEN

Esta investigación aplicada analiza el desarrollo y la verificación de PANVIZ IRAS, una aplicación web local para gestionar notificaciones sintéticas de infecciones respiratorias agudas. El estudio aborda un problema de Calidad de Software: transformar archivos CSV potencialmente incompletos o duplicados en resultados consolidados verificables, manteniendo trazabilidad entre requisitos, implementación, pruebas y cambios. Se empleó un estudio de caso técnico con revisión del repositorio, pruebas automatizadas y consulta de SQLite. El sistema utiliza Flask, Jinja2, SQLite y servicios separados de validación, consolidación y exportación. El conjunto inicial de veinte filas permite comprobar quince aceptaciones y cinco rechazos; una ampliación de cien filas válidas incrementó el total persistido a 115 notificaciones. En la ejecución de verificación realizada para esta investigación aprobaron 51 pruebas. Se identificó que las distribuciones uniformes de los gráficos respondían al conjunto inicial y no a un defecto de agregación. Los resultados sustentan la corrección funcional del piloto dentro de los escenarios evaluados, sin demostrar eficacia clínica, representatividad epidemiológica ni madurez organizacional CMMI.

Palabras clave: calidad de software, calidad de datos, Flask, SQLite, pruebas automatizadas, CMMI, trazabilidad, IRAS.

INTRODUCCIÓN

El valor de un sistema de información depende de la confiabilidad de los datos que recibe y de la posibilidad de explicar cómo produce sus resultados. En vigilancia respiratoria, una notificación duplicada o una fecha imposible altera los conteos si se incorpora sin control. La Organización Mundial de la Salud reconoce que el análisis de información de establecimientos debe considerar su calidad y propone revisiones periódicas e independientes de los datos reportados (OMS, s. f.). Esta investigación utiliza ese principio como motivación, sin presentar el prototipo como una herramienta sanitaria oficial.

PANVIZ IRAS se desarrolló en el curso Calidad de Software para estudiar la relación entre requisitos verificables, procesamiento de datos y evidencia de calidad. El sistema importa notificaciones, identifica rechazos, consolida por semana ISO, establecimiento, distrito y grupo etario, y permite exportar resultados reconciliados. El cambio CR01 incorpora el filtro por distrito como una modificación identificable del comportamiento funcional.

El aporte del trabajo consiste en vincular un producto ejecutable con un proceso documentado. La investigación examina tanto el código y los datos como la correspondencia entre requisito, prueba y versión. Las conclusiones se limitan al repositorio y a los conjuntos sintéticos observados. No se recopilaron datos de pacientes ni se realizaron intervenciones clínicas o pruebas con personal sanitario.

PLANTEAMIENTO DEL PROBLEMA

Descripción del problema

La entrada CSV facilita el intercambio de datos, pero no garantiza que los campos estén completos, que los identificadores sean únicos o que las fechas existan. Cuando estas condiciones no se comprueban, los errores se trasladan a cuadros y gráficos, y el usuario pierde la capacidad de reconciliar las cifras publicadas con los registros originales. La dificultad no se resuelve únicamente mediante una interfaz más atractiva: requiere reglas explícitas, integridad persistente y resultados contrastables.

En el caso académico, el problema comprende además la ausencia inicial de un flujo que conecte requisitos, código, pruebas y cambio controlado. Se necesita demostrar que una ampliación, como filtrar por distrito, conserva la coherencia de los resultados y no modifica silenciosamente el alcance de los datos exportados. La definición de criterios previos permite evaluar el comportamiento sin depender de apreciaciones subjetivas sobre la apariencia del dashboard.

Pregunta de investigación

¿Cómo implementar y verificar un sistema local de notificaciones IRAS que controle errores de entrada, consolide resultados reconciliados y mantenga trazabilidad de requisitos y cambios mediante prácticas seleccionadas de calidad de software?

Objetivo general

Desarrollar y evaluar técnicamente PANVIZ IRAS mediante validación de datos, persistencia transaccional, pruebas automatizadas y trazabilidad, tomando como referencia prácticas seleccionadas de CMMI DEV v1.3 en un contexto académico.

Objetivos específicos

Definir un contrato CSV y reglas verificables para los seis campos requeridos, junto con un catálogo etario didáctico y un conjunto de resultados esperados anterior a la ejecución.

Implementar importación con vista previa, validación por fila, almacenamiento en SQLite, consolidación por dimensiones y exportación con reconciliación de totales.

Verificar las categorías funcionales iniciales y el filtro por distrito CR01 mediante pruebas unitarias y de integración sobre bases temporales independientes.

Analizar las diferencias entre el conjunto mínimo de prueba y una ampliación sintética para explicar el comportamiento de las visualizaciones y sus límites interpretativos.

Documentar la relación entre requisitos, código, pruebas, proceso y evidencias, identificando las actividades humanas y organizacionales que permanecen pendientes.

Justificación y delimitación

El trabajo es pertinente para Calidad de Software porque convierte propiedades generales, como exactitud y mantenibilidad, en condiciones observables. Rechazar una fecha imposible, impedir la repetición de un identificador y obtener diferencia de reconciliación cero son verificaciones concretas. La separación de servicios facilita localizar reglas y probarlas sin depender de la presentación visual.

La investigación se delimita a una aplicación local y a información sintética. No incluye autenticación, administración de usuarios, clasificación clínica ni despliegue productivo. Los nombres de los establecimientos de la ampliación son de demostración; su distribución no expresa carga real de enfermedad. Tampoco se estima prevalencia, incidencia ni riesgo por distrito, porque no existen población de referencia ni procedimientos de captación representativos.

MARCO TEÓRICO

Calidad de datos y control de entrada

La calidad de datos requiere revisar completitud, consistencia y plausibilidad antes de interpretar agregados. La OMS plantea que los datos reportados por establecimientos deben someterse a procesos de aseguramiento y revisión; disponer de registros electrónicos no garantiza su confiabilidad (OMS, s. f.). En este piloto, el principio se operacionaliza con campos obligatorios, fechas válidas, identificadores únicos y catálogos explícitos. Estos controles cubren dimensiones acotadas y no sustituyen una auditoría sanitaria integral.

Es necesario distinguir un dato formalmente válido de uno verdadero. Una notificación puede cumplir el contrato y aun así no representar un evento real. PANVIZ IRAS comprueba estructura y coherencia de los datos sintéticos, pero no confirma diagnósticos. Por ello, el reporte de rechazo describe errores de calidad de datos y evita emitir conclusiones médicas.

Gestión de procesos y CMMI

El Software Engineering Institute presentó CMMI v1.3 como una versión de su conjunto de modelos, incluido CMMI para Desarrollo (SEI, 2010). La referencia histórica permite organizar prácticas relacionadas con requisitos, planificación, configuración, verificación y validación. En el estudio, estas prácticas se utilizan para ordenar las evidencias del ciclo de desarrollo, no para certificar el nivel de madurez de una organización.

La matriz del repositorio considera siete áreas de nivel 2 y once de nivel 3. Entre las primeras se encuentran REQM, PP, PMC, SAM, MA, PPQA y CM; entre las segundas, RD, TS, PI, VER, VAL, OPF, OPD, OT, IPM, RSKM y DAR. La documentación identifica estados parciales y actividades sin evidencia. La gestión de proveedores se analiza como no aplicable al alcance sin adquisición contratada, con justificación explícita.

La verificación comprueba correspondencia con requisitos; la validación examina si el producto resulta adecuado para el uso previsto. Una suite satisfactoria aporta evidencia de verificación dentro de los casos ejecutados, pero no acredita por sí misma aceptación del usuario, formación del equipo o revisión por pares. Esta distinción guía la interpretación de los resultados.

Arquitectura modular con Flask

La documentación oficial de Flask describe el patrón de fábrica de aplicaciones como un mecanismo para crear instancias con configuraciones distintas, especialmente útil para pruebas (Pallets Projects, s. f.). PANVIZ IRAS utiliza create_app y Blueprints para separar las rutas del dashboard, importación, validación y reportes. Esta organización permite sustituir la base de demostración por una temporal sin modificar la lógica funcional.

Las plantillas Jinja2 presentan los resultados y los servicios concentran las reglas. Esta separación evita que un gráfico contenga cifras de demostración ajenas al estado persistido. Chart.js transforma agregados recibidos en visualizaciones; el conteo se obtiene de las notificaciones aceptadas, no de valores constantes escritos en JavaScript.

Persistencia e integridad con SQLite

SQLite documenta transacciones con propiedades de atomicidad, consistencia, aislamiento y durabilidad; los cambios de una transacción se completan conjuntamente o no se incorporan (SQLite, s. f.). La aplicación aprovecha esta capacidad para almacenar el lote, las notificaciones aceptadas y los rechazos de una confirmación como una unidad lógica.

La integridad se refuerza con una clave primaria de notificación y referencias al lote. La confirmación vuelve a validar los identificadores bajo una transacción de escritura, dado que otra carga podría haber modificado los datos después de la vista previa. Esta decisión resuelve un riesgo que no se controla mediante validación en el navegador.

Pruebas y organización temporal

Pytest permite definir fixtures reutilizables para preparar contextos y dependencias de las pruebas (pytest development team, s. f.). El proyecto emplea clientes Flask y bases temporales para aislar escenarios. La evaluación no depende de que la base usada para la demostración permanezca vacía o conserve exactamente el conjunto inicial.

La agrupación semanal utiliza el calendario ISO mediante date.isocalendar. La documentación de Python distingue el año ISO del año calendario, lo que evita asignaciones incorrectas en fechas próximas al cambio de año (Python Software Foundation, s. f.). El sistema conserva año y semana en una misma clave, por ejemplo 2026-W02.

METODOLOGÍA

Enfoque y diseño

Se realizó una investigación aplicada de alcance descriptivo y evaluativo, mediante un estudio de caso técnico del repositorio PANVIZ IRAS. La unidad de análisis es el flujo de importación, validación, consolidación y exportación. El procedimiento comprende revisión de requisitos, inspección de componentes, ejecución de pruebas y contraste de registros persistidos con resultados esperados.

No se utilizó un experimento clínico ni una muestra poblacional. Los resultados se obtuvieron de conjuntos sintéticos diseñados para ejercitar reglas y presentación. La ampliación aleatoria introduce variación funcional, pero no constituye un muestreo estadístico de notificaciones reales. Su utilidad es observar cómo responde el sistema ante cantidades diferentes por semana y distrito.

Fuentes e instrumentos

La fuente interna comprende el código de src/panviz, los CSV de datos/entrada, el oráculo JSON, las pruebas de tests, la documentación de requisitos y los registros de evidencias. Las fuentes externas son documentación oficial de OMS, SEI, Flask, SQLite, Python, pytest y W3C. Las referencias técnicas explican decisiones; los resultados del producto se sustentan en observaciones del repositorio.

Los instrumentos fueron la suite pytest, consultas agregadas de SQLite, comparación de totales y revisión de la matriz de trazabilidad. La ejecución de pruebas consultada en esta investigación produjo 51 aprobaciones. El registro existente verificacion.json también conserva las versiones del entorno y el resultado del conjunto inicial. Las aprobaciones humanas indicadas como pendientes no se convierten en resultados aprobados.

Procedimiento de desarrollo y evaluación

Primero se definieron los requisitos R01 a R05 y el contrato de datos. Después se implementaron la fábrica Flask, los Blueprints y los servicios. Se preparó un conjunto mínimo con errores deliberados y se estableció el oráculo. Finalmente se incorporó CR01 y se amplió la demostración con cien registros nuevos. La investigación contrastó el estado final de los datos con las agregaciones usadas por el dashboard.

La evaluación funcional incluye importación válida, campo obligatorio vacío, fecha imposible, duplicado, consolidación conocida y reconciliación de la exportación. CR01 añade la comprobación del distrito seleccionado. Los escenarios adicionales examinan formatos inválidos, límites, reimportación, confirmación repetida, vista previa desactualizada, protección de formularios y cambio del año ISO.

Criterios e indicadores

La aceptación inicial exige quince filas válidas aceptadas y cinco inválidas rechazadas sin falsos rechazos. La detección de errores se expresa como inválidas detectadas entre cinco, multiplicado por cien. La reconciliación se expresa como total consolidado menos aceptados bajo los mismos filtros, con objetivo cero. El porcentaje de pruebas satisfactorias se obtiene de aprobadas entre ejecutadas.

La trazabilidad se revisa para R01, R02, R03, R04, R05 y CR01. Un enlace existente a código y prueba es evidencia estructural; su aprobación y suficiencia requieren revisión nominativa. La formación, el seguimiento y la validación de usuario se analizan como actividades del proceso distintas de la ejecución automatizada.

DESARROLLO DEL SISTEMA

Requisitos y contrato CSV

R01 establece la importación de id, fecha_notificacion, establecimiento, distrito, grupo_etario y evento. El formato admite UTF-8 con marca BOM opcional, coma o punto y coma y columnas en cualquier orden, siempre que se conserve el contrato. Se limita la carga a dos megabytes y diez mil registros. Un archivo sin datos o con columnas repetidas no se acepta como carga válida.

R02 establece el rechazo por campos vacíos, fechas imposibles y duplicados, junto con el número de fila y el motivo. La aplicación añade controles de longitud, caracteres de control, formato del identificador, catálogo etario y evento. El catálogo didáctico incluye 0-4, 5-11, 12-17, 18-29, 30-59 y 60+. El único evento permitido es IRAS.

R03 define la consolidación por año y semana ISO, establecimiento, distrito y grupo etario. R04 exige exportar y reconciliar los resultados. R05 mantiene enlaces entre requisitos, diseño, código, pruebas y versión. El catálogo y los resultados esperados están documentados antes de su utilización como criterio de aceptación.

Modelo de datos y flujo de importación

La tabla batches identifica los lotes y conserva archivo, origen, fecha y contadores. notifications almacena las aceptaciones y su lote; rejections registra fila, identificador, motivo y datos normalizados; previews conserva temporalmente el archivo asociado a una confirmación. No se requiere un servicio separado de base de datos.

El usuario carga un archivo y revisa una vista previa que todavía no incorpora notificaciones al conjunto aceptado. Al confirmar, el servidor vuelve a comprobar los identificadores y guarda el resultado bajo una transacción. Una vista previa vence después de una hora y no debe reutilizarse. Los rechazos permanecen consultables para corregir el archivo de origen.

La interfaz permite seleccionar un archivo y revisar su contenido antes de confirmar. El historial identifica el lote de cien registros adicionales.

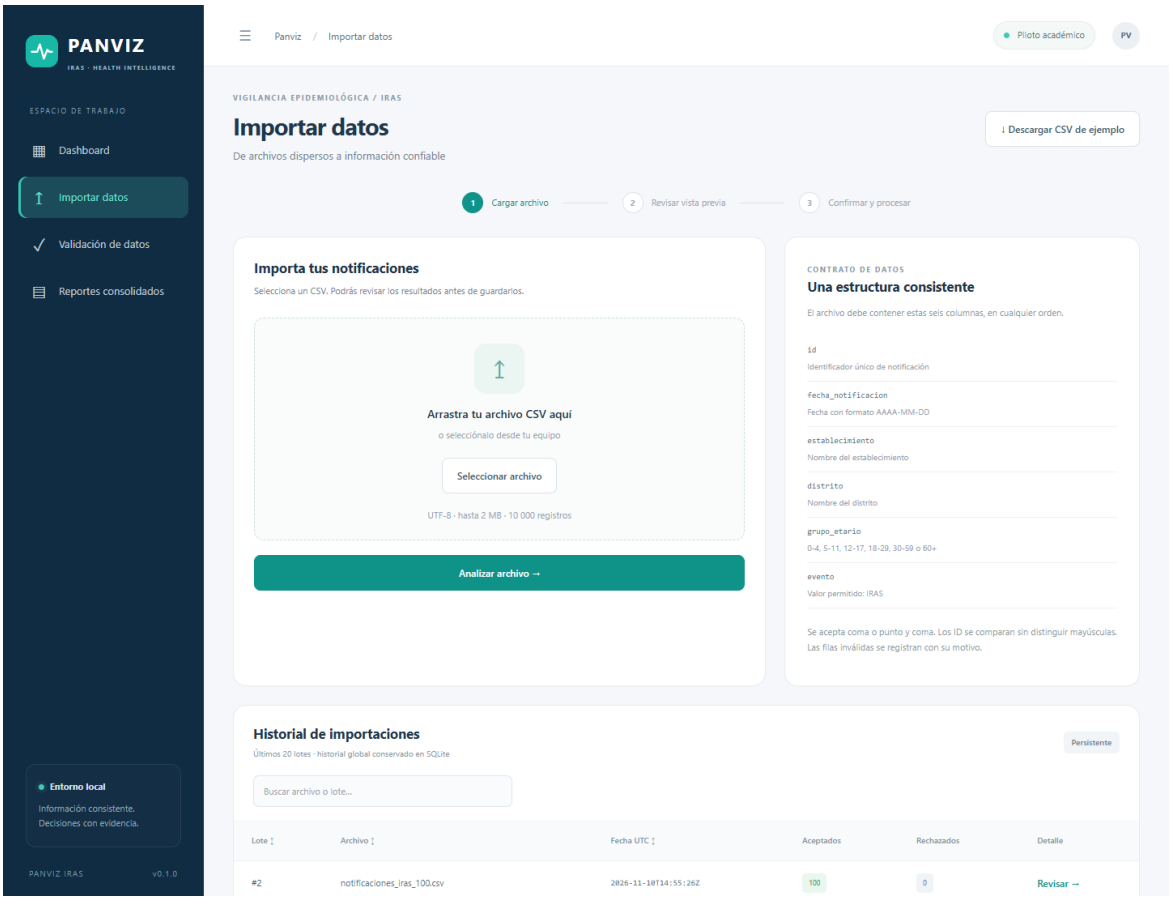


Figura 1. Pantalla de importación y contrato de datos CSV. Fuente: captura de las páginas generadas por Flask con datos de SQLite local.

La repetición de un archivo no significa que sus datos se sumen nuevamente. Los identificadores aceptados se rechazan en cargas posteriores. Si se vuelve a importar el conjunto inicial ya persistido, el resultado esperado es cero aceptados y veinte rechazos: quince por duplicidad y cinco por las infracciones iniciales. Esta condición se diferencia del oráculo de una base vacía.



Figura 2. Flujo implementado en PANVIZ IRAS. Fuente: elaboración a partir del código del repositorio.

Consolidación y exportación

Las consultas aplican los filtros antes de agrupar y calculan el número de notificaciones de cada combinación dimensional. La exportación comparte los filtros de la pantalla. Antes de generar el CSV, el sistema verifica que la suma de los grupos coincida con las aceptaciones de la selección. Los rechazos se exportan por separado con información de lote y motivo.

Las celdas potencialmente interpretables como fórmulas reciben protección al exportarse. Las consultas utilizan parámetros para los valores de filtros y una lista cerrada de dimensiones. Estas decisiones reducen riesgos concretos, aunque no constituyen una evaluación integral de seguridad de una aplicación productiva.

Pantallas y accesibilidad

El dashboard presenta indicadores, tendencia semanal, distribución territorial y registros recientes. Importación permite arrastrar o seleccionar archivos y confirmar el procesamiento; Validación muestra observaciones; Reportes presenta grupos y descargas. El menú es plegable y las tablas permiten búsqueda y ordenamiento en el navegador.

La vista resume el conjunto sintético persistido. Los indicadores y gráficos se calculan a partir de las notificaciones aceptadas.

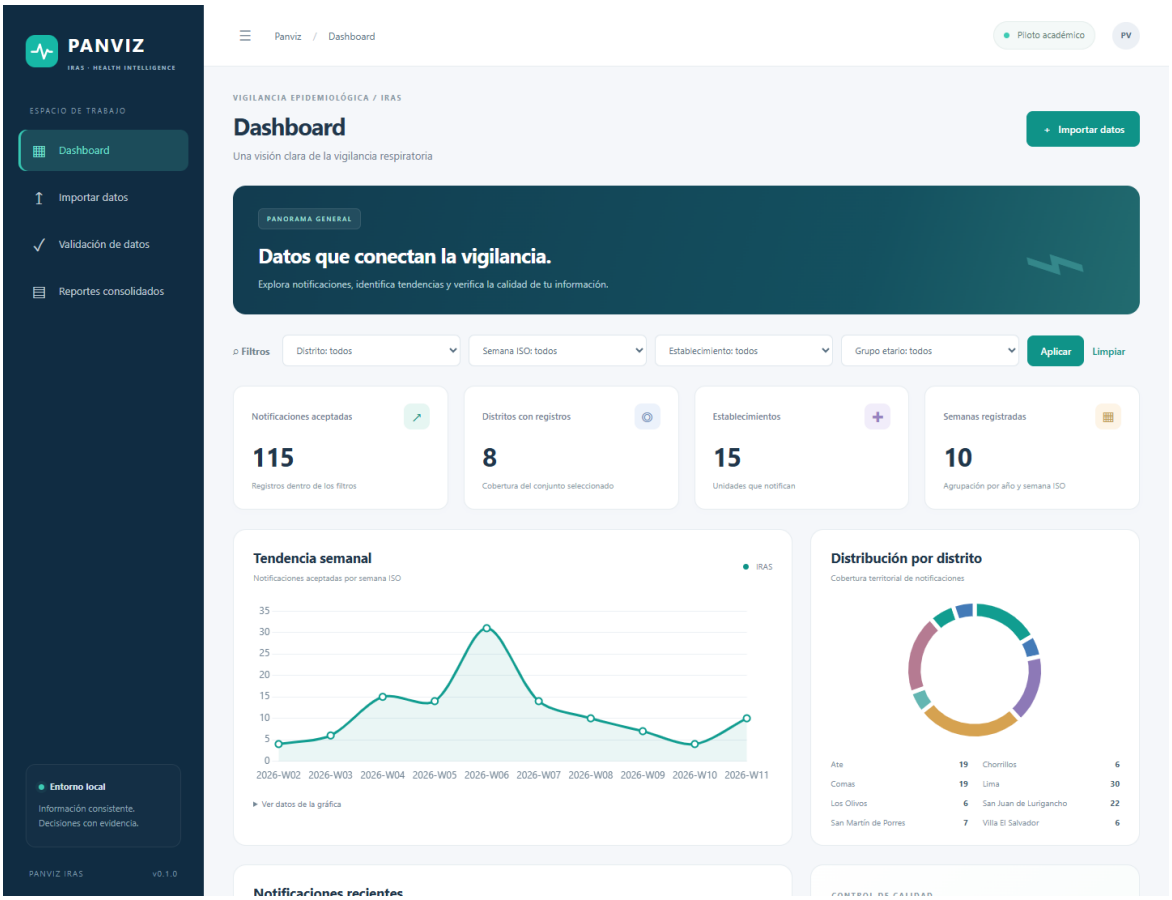


Figura 3. Dashboard con 115 notificaciones aceptadas, ocho distritos y diez semanas. Fuente: captura de las páginas generadas por Flask con datos de SQLite local.

La búsqueda visual de una tabla no modifica los filtros usados para exportar; esta diferencia se informa en pantalla. El proyecto incorpora etiquetas, navegación por teclado, enlace para saltar al contenido y datos tabulados alternativos a la gráfica. Estos controles se relacionan con objetivos de accesibilidad de WCAG 2.2, pero no equivalen a una certificación de conformidad (W3C, 2024).

Cambio controlado CR01

CR01 solicita filtrar el consolidado por distrito. En el conjunto inicial, seleccionar Ate debe producir cinco aceptaciones y una exportación de total cinco; limpiar el filtro debe restaurar quince. El distrito inexistente debe devolver cero sin alterar la base. El criterio utiliza cifras previas a la ampliación y permanece válido en las pruebas aisladas.

La funcionalidad se identifica mediante DISTRICT_FILTER_ENABLED, consultas compartidas y pruebas específicas. El perfil baseline deshabilita la aplicación del distrito en las rutas HTTP. La referencia funcional B0 facilita comparar comportamientos, pero no se presenta como una aprobación histórica del equipo ni sustituye los controles de configuración requeridos.

Este resultado corresponde al estado ampliado de 115 registros. Se distingue del oráculo inicial de cinco notificaciones por distrito.

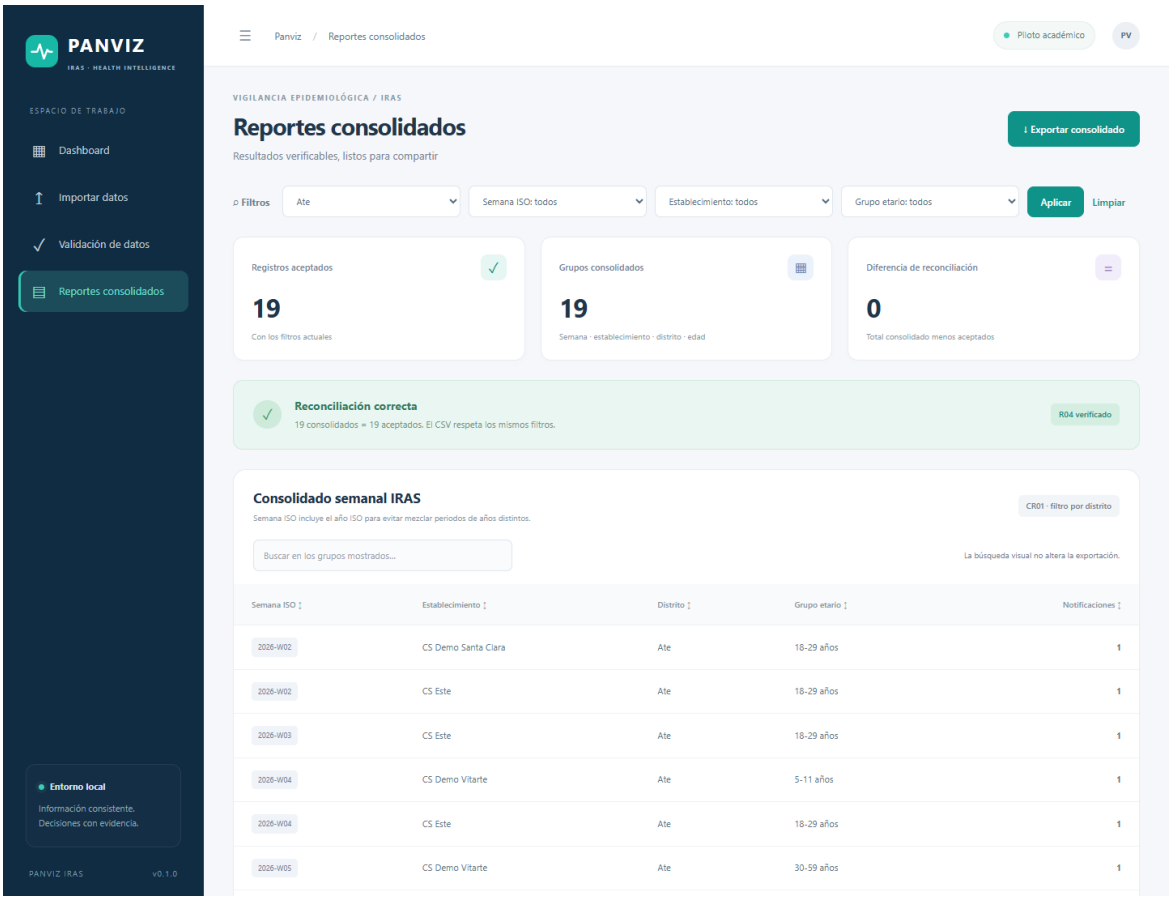


Figura 4. Consolidado de Ate con diecinueve aceptaciones y diferencia de reconciliación cero. Fuente: captura de las páginas generadas por Flask con datos de SQLite local.

RESULTADOS

Resultado del conjunto mínimo

El conjunto inicial contiene veinte registros: quince válidos y cinco inválidos deliberados. Los rechazos corresponden a un identificador repetido, un distrito vacío, una fecha imposible, un grupo etario fuera del catálogo y un evento distinto de IRAS. Sus líneas de origen son 17 a 21, contando el encabezado. El resultado registrado es quince aceptaciones y cinco rechazos.

La captura muestra los cinco motivos deliberados del conjunto mínimo: duplicado, campo vacío, fecha imposible, grupo etario y evento inválidos.

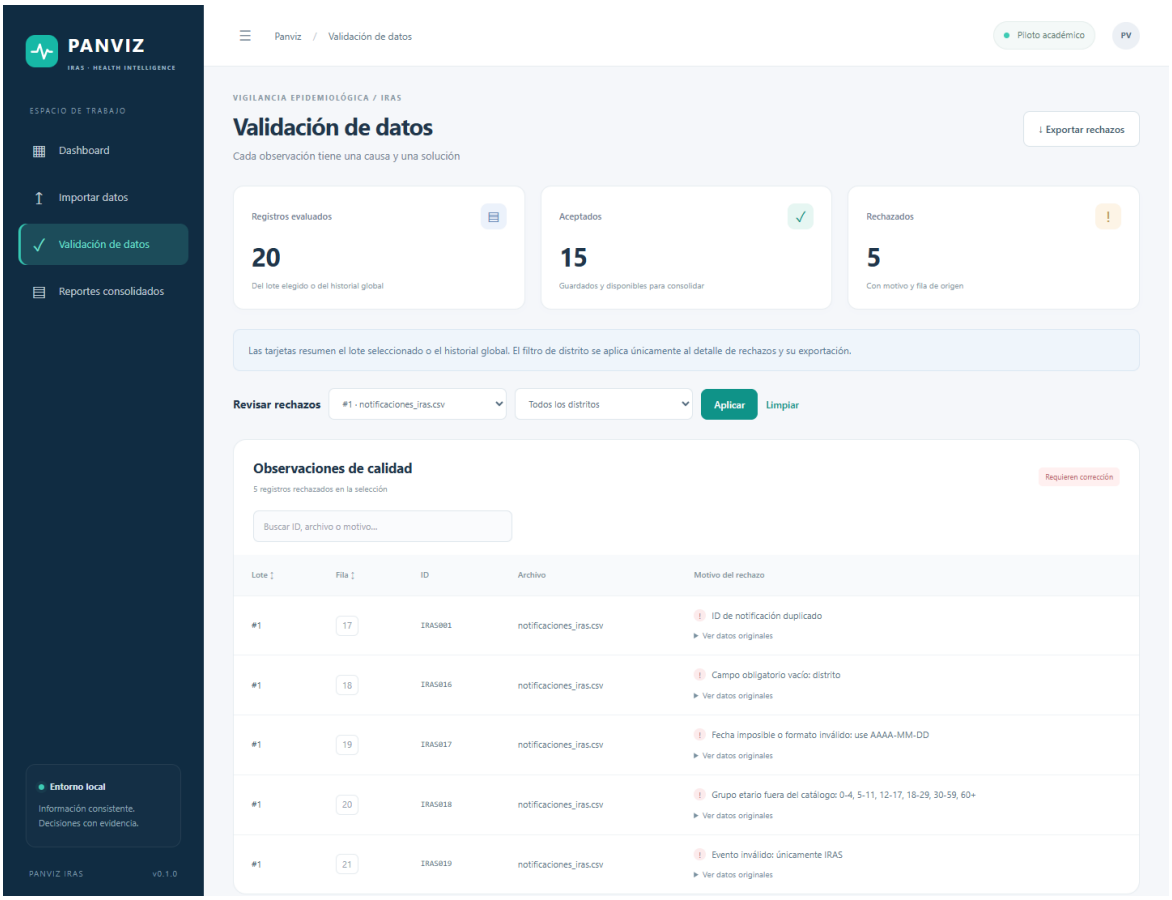


Figura 5. Validación del lote inicial con quince aceptaciones y cinco rechazos. Fuente: captura de las páginas generadas por Flask con datos de SQLite local.

La detección de errores alcanza cinco entre cinco, es decir, cien por ciento en ese conjunto. El total consolidado es quince y la diferencia de reconciliación es cero. Los tres distritos iniciales tienen cinco aceptaciones cada uno y las cinco semanas contienen tres. Esa uniformidad fue una propiedad de los datos de contraste, no un resultado epidemiológico.

Verificación automatizada

La ejecución realizada para la investigación finalizó con 51 pruebas aprobadas y ninguna fallida, equivalente a cien por ciento de las pruebas ejecutadas en esa sesión. El resultado corresponde al entorno local y a los escenarios implementados. No se interpreta como ausencia universal de defectos ni como cobertura total de todas las entradas posibles.

Las pruebas verifican tanto reglas aisladas como integración de rutas, plantillas y exportaciones. También comprueban que los enlaces de la matriz de trazabilidad apunten a archivos existentes. Esta última comprobación detecta referencias rotas, pero no sustituye una revisión de la pertinencia del diseño o una aprobación independiente del producto.

Ampliación de datos sintéticos

Para evitar una demostración visual uniforme, se generaron cien registros adicionales con identificadores nuevos, ocho distritos, doce establecimientos de demostración y los seis grupos etarios. Las fechas se ordenaron y se distribuyeron en diez semanas con frecuencias desiguales. La carga directa autorizada se realizó con validación previa, transacción y copia de seguridad; aceptó cien registros y no generó rechazos.

El estado persistido final contiene 115 aceptaciones. Los cinco rechazos originales se conservan asociados al primer lote. La ampliación no reemplaza el conjunto mínimo ni modifica su oráculo: constituye un segundo conjunto destinado a demostrar variación y persistencia. El archivo de entrada y la evidencia de importación permiten identificar su procedencia.

La distribución final por distrito es Lima, 30; San Juan de Lurigancho, 22; Ate, 19; Comas, 19; San Martín de Porres, 7; Chorrillos, 6; Los Olivos, 6; y Villa El Salvador, 6. La suma es 115. Las diferencias reflejan la generación sintética y no comparaciones reales de salud entre jurisdicciones.

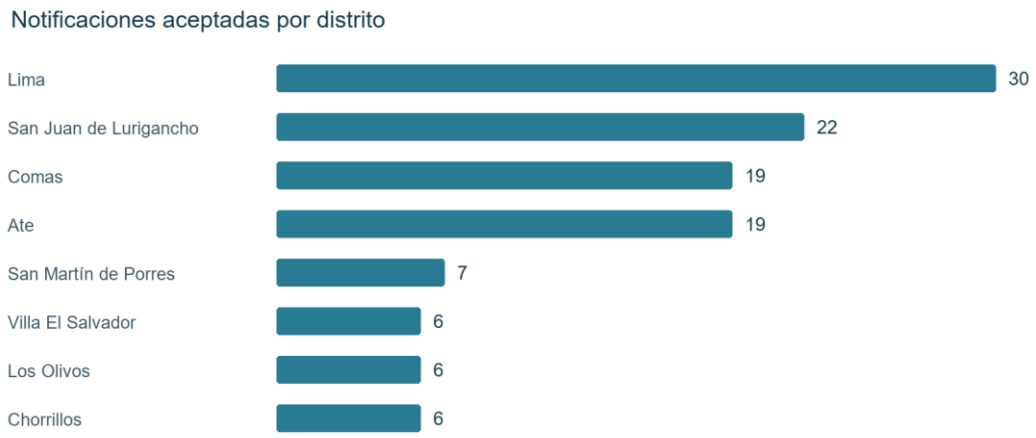


Figura 6. Distribución de las 115 notificaciones sintéticas aceptadas por distrito. Fuente: SQLite local.

La tendencia final por semanas ISO W02 a W11 de 2026 contiene, respectivamente, 4, 6, 15, 14, 31, 14, 10, 7, 4 y 10 notificaciones. La suma vuelve a ser 115. La mayor frecuencia del conjunto ocurre en W06 con 31 notificaciones; describirla como pico de la demostración es válido, mientras que denominarla brote real no lo sería.

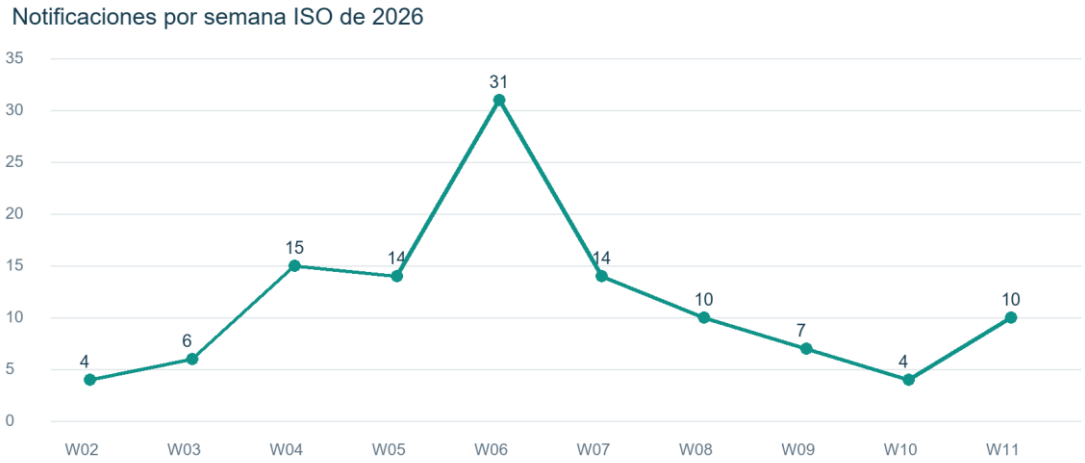


Figura 7. Tendencia semanal del conjunto sintético. Fuente: SQLite local; año ISO 2026.

Diagnóstico de las gráficas uniformes

La revisión del código mostró que el dashboard cuenta las notificaciones por semana y distrito, y que Chart.js utiliza esos agregados. Al consultar la base antes de la carga adicional se encontraron solo quince aceptaciones, repartidas de forma idéntica. La existencia del CSV de cien filas en la carpeta no implicaba su incorporación a SQLite.

La corrección consistió en persistir el nuevo conjunto y comprobar los totales, conservando el cálculo de los gráficos. Este hallazgo evidencia una diferencia entre archivo generado, archivo importado y dato mostrado. La verificación del estado persistido permitió explicar el comportamiento sin introducir números artificiales en la presentación.

DISCUSIÓN

La evidencia respalda la viabilidad de un flujo local que detecta las infracciones previstas, conserva lotes y produce resultados reconciliados. El vínculo entre contrato, prueba y exportación facilita revisar por qué una fila aparece o no en un agregado. En este sentido, la calidad se expresa en propiedades comprobables y no solamente en la apariencia de las pantallas.

La uniformidad inicial ilustra un riesgo metodológico: un conjunto pequeño y balanceado puede ocultar problemas de interpretación visual o aparentar que las agregaciones son constantes. La ampliación ayuda a observar variación, pero no demuestra robustez ante grandes volúmenes ni representatividad de los distritos. Su valor es funcional y exploratorio.

La suite satisfactoria fortalece la evidencia de verificación dentro del alcance, pero no resuelve las brechas de proceso. La aceptación funcional por E2, la revisión independiente por E4, la formación y las aprobaciones de baselines siguen requiriendo registros personales. Tampoco una carpeta de documentación demuestra adopción sostenida del proceso por una organización.

El uso de SQLite es congruente con una demostración local sin servidor adicional. La decisión equilibra simplicidad, persistencia e integridad; no establece que sea la mejor alternativa para cualquier sistema sanitario. Un despliegue multiusuario con mayor concurrencia, control de acceso y datos reales necesitaría una nueva evaluación técnica y organizacional.

LIMITACIONES Y CONSIDERACIONES ÉTICAS

El estudio se limita a datos sintéticos y a un único repositorio. No se recogieron opiniones de usuarios ni mediciones de rendimiento, usabilidad o cobertura de código. El número de pruebas indica casos ejecutados, no porcentaje de código cubierto. No se dispone de evidencia suficiente para atribuir mejoras cuantitativas de productividad o reducción de errores en una institución real.

Los nombres geográficos no convierten los registros en estadísticas oficiales. No deben usarse para decisiones clínicas ni asignación real de recursos. La ausencia de identificadores personales reduce la exposición de información sensible en la demostración, pero una eventual adopción real exigiría gobernanza, privacidad, autorización y revisión de los requisitos aplicables.

El trabajo conserva pendientes visibles y evita registrar firmas, asistencia o aprobaciones inexistentes. Los resultados de ejecución se distinguen de los resultados esperados. La investigación utiliza CMMI DEV v1.3 como marco académico histórico y no declara certificación ni madurez organizacional alcanzada.

CONCLUSIONES

PANVIZ IRAS implementa un flujo funcional de importación, validación, consolidación y exportación con SQLite. La estructura modular permite localizar las reglas de negocio y ejecutarlas en pruebas independientes de la base de demostración.

El conjunto inicial permitió verificar quince aceptaciones y cinco rechazos deliberados, con reconciliación de diferencia cero. En la sesión de evaluación aprobaron 51 pruebas. Estos resultados sustentan la corrección observada en los escenarios evaluados y deben interpretarse dentro de ese alcance.

La incorporación de cien registros adicionales elevó las aceptaciones persistidas a 115 y produjo distribuciones variables. El diagnóstico de las gráficas identificó que la igualdad original provenía de datos balanceados y de una ampliación aún no importada, por lo que no fue necesario alterar el algoritmo de agregación.

CR01 se vincula a criterios, código y pruebas específicos. La documentación y el perfil B0 contribuyen a la trazabilidad funcional, mientras que las revisiones, aprobaciones y contribuciones del equipo deben completarse con evidencia real.

La aplicación y sus evidencias constituyen un piloto académico de Calidad de Software. Su continuidad debe orientarse a validación con usuarios, revisión independiente, reproducción desde copia limpia y seguimiento de las brechas de proceso, sin extrapolar los resultados a eficacia sanitaria o madurez CMMI.

RECOMENDACIONES

Completar la revisión cruzada, la validación funcional y las actas con nombres, fechas y decisiones reales. Conservar tanto los resultados satisfactorios como las incidencias de ejecuciones anteriores.

Repetir la demostración desde una copia limpia y contrastar el conjunto inicial en una base temporal. Mantener separados el oráculo de veinte filas y la ampliación de cien para no confundir criterios de aceptación con datos de presentación.

Incorporar futuras mediciones de rendimiento y accesibilidad con criterios previos, así como escenarios de archivos de mayor tamaño y uso concurrente. Toda ampliación debe evaluarse mediante un cambio registrado y regresión del comportamiento existente.

Reutilizar el proceso en otro módulo, reunir evidencia durante varios ciclos y revisar las brechas de las dieciocho áreas antes de plantear una evaluación organizacional. La continuidad debe establecer responsables y evidencia esperada en lugar de prometer un nivel de madurez por una única entrega.

REFERENCIAS

Organización Mundial de la Salud. (s. f.). Data quality assurance DQA. Recuperado el 10 de noviembre de 2026, de https://www.who.int/data/data-collection-tools/health-service-data/data-quality-assurance-dqa

Pallets Projects. (s. f.). Application factories. Flask documentation. Recuperado el 10 de noviembre de 2026, de https://flask.palletsprojects.com/en/stable/patterns/appfactories/

Python Software Foundation. (s. f.). datetime Basic date and time types. Python documentation. Recuperado el 10 de noviembre de 2026, de https://docs.python.org/3/library/datetime.html

pytest development team. (s. f.). How to use fixtures. pytest documentation. Recuperado el 10 de noviembre de 2026, de https://docs.pytest.org/en/stable/how-to/fixtures.html

Software Engineering Institute. (2010, 28 de octubre). Software Engineering Institute releases version 1.3 of CMMI product suite. Carnegie Mellon University. https://www.sei.cmu.edu/news/software-engineering-institute-releases-version-13-of-cmmi-product-suite/

SQLite. (s. f.). SQLite is transactional. Recuperado el 10 de noviembre de 2026, de https://www.sqlite.org/transactional.html

World Wide Web Consortium. (2024, 12 de diciembre). Web Content Accessibility Guidelines WCAG 2.2. https://www.w3.org/TR/2024/REC-WCAG22-20241212/

Equipo PANVIZ IRAS. (2026). PANVIZ IRAS System [Código fuente y documentación del proyecto académico]. https://github.com/AnthoLogy63/PANVIZ_IRAS-System

ANEXO DOCUMENTAL

La evidencia interna comprende datos/entrada/oraculo.json, notificaciones_iras.csv y notificaciones_iras_100.csv; evidencias/verificacion.json; evidencias/importacion-datos-variados.json; docs/requisitos/trazabilidad.csv; docs/gestion/CR01.md; y los módulos de tests. Estas rutas permiten localizar el origen de los resultados expuestos.

Los datos de validación inicial y ampliación se diferencian mediante su lote. Las revisiones y la liberación se documentan en evidencias/revision.md, validacion.md y liberacion.md. Los estados pendientes de dichos archivos no se interpretan como actividades aprobadas.